

Universidad Técnica Estatal De Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Ingeniería en Software (Rediseño)

Aplicaciones Web

2025-2026 SPA

Práctica Experimental Unidad II

EQUIPO I

ARIAS MOREIRA MAYBELIN GREGORIA

CASTRO PALMA JUSTYN MOISES

RIOS CUYABAZO JHON KEVIN

Dr. Gleiston Cicerón Guerrero Ulloa. Ing.

Viernes 19 de junio 2026

URL

<https://github.com/may10311999/GA----Pr-ctica-Experimental-Unidad-II/tree/main>

1. INTRODUCCION

El desarrollo de aplicaciones web dinámicas ha evolucionado significativamente en las últimas décadas, dando lugar a múltiples tecnologías de servidor que compiten en el mercado. La presente práctica experimental aborda el estudio comparativo de dos tecnologías ampliamente utilizadas: PHP (Hypertext Preprocessor) y ASP.NET Core de Microsoft, aplicadas al desarrollo de un sistema de gestión de tareas con autenticación completa y operaciones CRUD.

El objetivo principal de este trabajo es implementar una aplicación web dinámica que cumpla con los estándares de seguridad OWASP, específicamente en la prevención de XSS (Cross-Site Scripting), protección CSRF (Cross-Site Request Forgery), almacenamiento seguro de contraseñas y cabeceras de seguridad HTTP. Ambas implementaciones fueron desarrolladas utilizando buenas prácticas de programación, incluyendo el uso de prepared statements o equivalentes para prevenir inyección SQL.

El informe se estructura en cinco secciones: introducción, desarrollo (fundamento teórico y resultados), tabla comparativa, conclusiones y referencias bibliográficas. Se incluye además el código fuente completo en un repositorio Git con los commits correspondientes.

2. DESARROLLO

2.1 Fundamento Teórico

2.1.1 Aplicaciones Web Dinámicas

Las aplicaciones web dinámicas son aquellas que generan contenido personalizado en tiempo real, interactuando con bases de datos y adaptándose a las acciones del usuario. A diferencia de las páginas estáticas, estas aplicaciones utilizan lenguajes de programación del lado del servidor para procesar solicitudes, consultar bases de datos y generar respuestas HTML personalizadas.

2.1.2 PHP (Hypertext Preprocessor)

PHP es un lenguaje de programación de código abierto ampliamente utilizado para el desarrollo web del lado del servidor. Según W3Techs (2025), más del 75% de los sitios web con tecnología de servidor conocida utilizan PHP. Su popularidad se debe a su facilidad de aprendizaje, extensa documentación y amplio ecosistema de frameworks como Laravel, Symfony y CodeIgniter[1].

Para el almacenamiento de contraseñas, PHP proporciona la función `password_hash()` que implementa algoritmos de hashing seguros como BCrypt, Argon2i y Argon2id. Para la prevención de XSS, se utiliza `htmlspecialchars()` con codificación UTF-8. Los prepared statements en PDO (PHP Data Objects) ofrecen protección contra inyección SQL al separar la estructura SQL de los datos del usuario.

PHP utiliza la extensión PDO (PHP Data Objects) para conectarse a diferentes sistemas de bases de datos. En este proyecto se empleó PDO_PGSQL para la conexión a PostgreSQL 18, aprovechando la portabilidad que ofrece PDO para migrar entre motores de base de datos sin cambiar la lógica de negocio.

2.1.3 ASP.NET Core

ASP.NET Core es un framework de código abierto desarrollado por Microsoft para la creación de aplicaciones web modernas y de alto rendimiento. Es multiplataforma, funcionando en Windows, Linux y macOS. ASP.NET Core incluye mecanismos de seguridad integrados como la protección CSRF mediante tokens AntiForgery, el codificado automático de salidas en Razor para prevenir XSS, y el PasswordHasher integrado en ASP.NET Core Identity para el almacenamiento seguro de contraseñas[2],[3].

Entity Framework Core es el ORM (Object-Relational Mapper) oficial de Microsoft que permite trabajar con bases de datos utilizando objetos .NET, eliminando la necesidad de escribir SQL directamente y previniendo inyecciones SQL de forma inherente.

2.1.4 Seguridad OWASP

El Open Web Application Security Project (OWASP) es una organización sin fines de lucro dedicada a mejorar la seguridad del software. Su OWASP Top 10 clasifica los riesgos de seguridad más críticos en aplicaciones web (OWASP Foundation, 2024)[4],[5]:

- a) Prevención XSS: El Cross-Site Scripting ocurre cuando un atacante inyecta scripts maliciosos en páginas web. Para prevenirlo, se debe sanear toda salida de datos no confiables.
- b) Token CSRF: Los ataques de Cross-Site Request Forgery fuerzan a un usuario autenticado a ejecutar acciones no deseadas. Los tokens CSRF únicos por sesión mitigan este riesgo.
- c) Almacenamiento de contraseñas: Las contraseñas deben almacenarse utilizando funciones de hashing adaptativas como BCrypt, con un factor de costo adecuado.

d) Cabeceras de seguridad HTTP: Cabeceras como X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security y Content-Security-Policy protegen contra diversos ataques

2.2 RESULTADOS

2.2.1 Implementación en PHP

La implementación en PHP se estructuró siguiendo el patrón MVC (Modelo-Vista-Controlador) con las siguientes carpetas:

config/: Configuración de base de datos PostgreSQL y funciones de seguridad.
modelos/: Clases Usuario y Tarea con prepared statements PDO.
controladores/: AuthController (registro, login, logout) y TareaController (CRUD completo).
vistas/: Plantillas HTML con sanitización de salidas.
public/: Punto de entrada único (front controller) y archivos estáticos.

Características de seguridad implementadas:

XSS: Uso de htmlspecialchars() con ENT_QUOTES y UTF-8 en todas las salidas.
CSRF: Generación de tokens únicos por sesión con bin2hex(random_bytes(32)).
Contraseñas: password_hash() con PASSWORD_BCRYPT y costo 12.
Cabeceras HTTP: X-Frame-Options, X-Content-Type-Options, HSTS, CSP, etc.
SQL Injection: Prepared statements PDO con bindParam().

2.2.2 Migración de SQLite a PostgreSQL

Durante el desarrollo del proyecto se realizó una migración del motor de base de datos de SQLite a PostgreSQL 18. SQLite se utilizó inicialmente por su simplicidad (base de datos embebida en un archivo), pero se migró a PostgreSQL para cumplir con los requisitos académicos de utilizar un sistema de gestión de bases de datos cliente-servidor.

Los cambios realizados fueron:

Conexión: Cambio de PDO("sqlite:ruta") a PDO("pgsql:host=...;dbname=...") con credenciales.
Fechas: SQLite usa datetime('now'), PostgreSQL usa CURRENT_TIMESTAMP.
Autoincremento: SQLite usa INTEGER PRIMARY KEY AUTOINCREMENT, PostgreSQL usa SERIAL.
Gestión: Se utilizó pgAdmin 4 como interfaz gráfica para administrar la base de datos.

La migración demostró la ventaja de usar PDO: al ser una capa de abstracción, solo fue necesario modificar la cadena de conexión y algunas funciones SQL específicas, sin alterar la lógica de negocio.

2.2.3 Implementación en ASP.NET Core

La implementación en ASP.NET Core se realizó con el framework MVC y Entity Framework Core:

Models/: Entidad Tarea con DataAnnotations para validación.
Data/: ApplicationDbContext con IdentityDbContext y configuración de entidades.
Controllers/: AccountController (autenticación Identity) y TareaController (CRUD).
Views/: Vistas Razor con @Html.AntiForgeryToken() y etiquetas HTML helper.

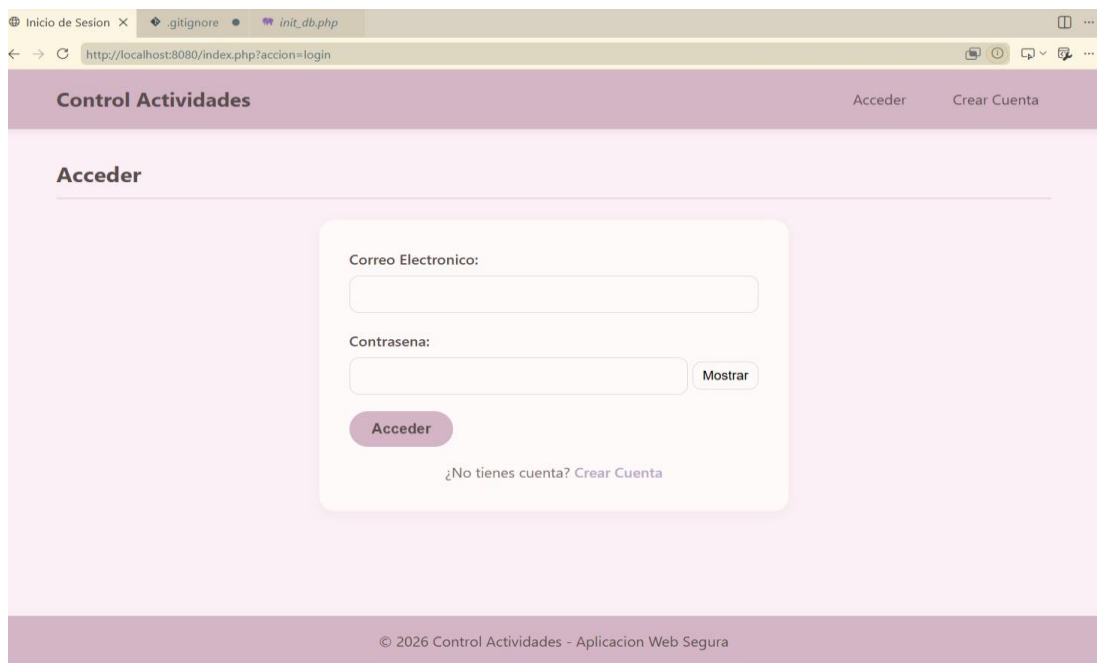
Características de seguridad:

XSS: Codificación automática de Razor (@ sintaxis).
CSRF: @Html.AntiForgeryToken() y [ValidateAntiForgeryToken] en controllers.
Contraseñas: PasswordHasher de ASP.NET Core Identity (PBKDF2).
Cabeceras HTTP: Middleware personalizado con las mismas cabeceras que PHP.
SQL Injection: Entity Framework Core con LINQ (parametrización automática).

3. EVIDENCIAS DEL FUNCIONAMIENTO DE LA APLICACIÓN WEB

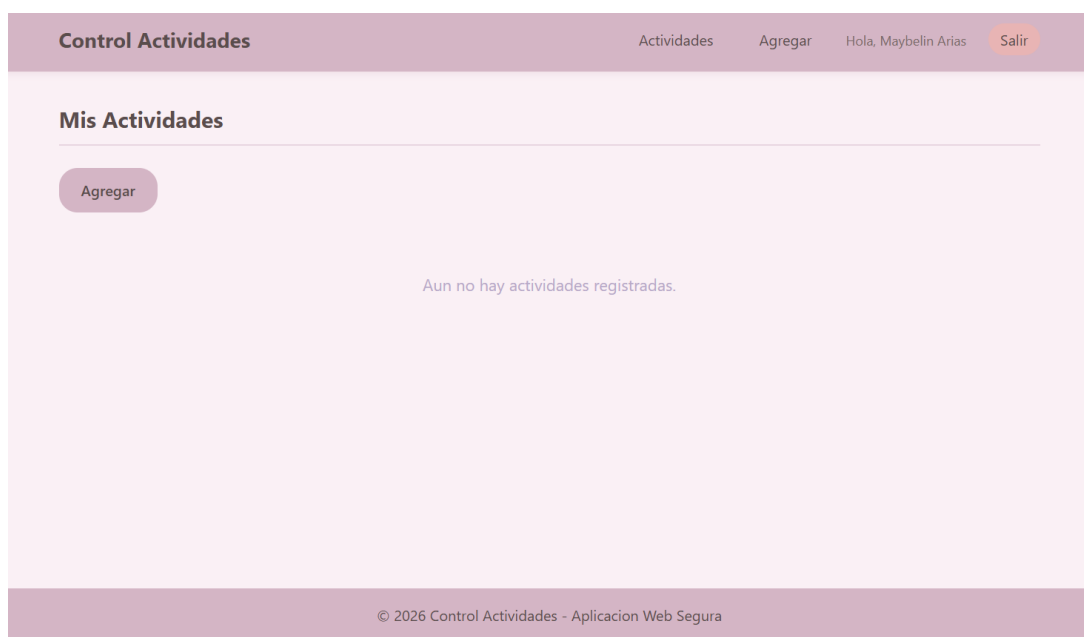
En esta sección se presentan las capturas de pantalla correspondientes a la aplicación desarrollada, con el fin de evidenciar el funcionamiento de las diferentes interfaces y las operaciones implementadas en el sistema.

A continuación, se presentan las capturas de pantalla correspondientes a la aplicación desarrollada, con el propósito de evidenciar la interfaz y verificar el correcto funcionamiento del sistema de autenticación y las diferentes funcionalidades implementadas.



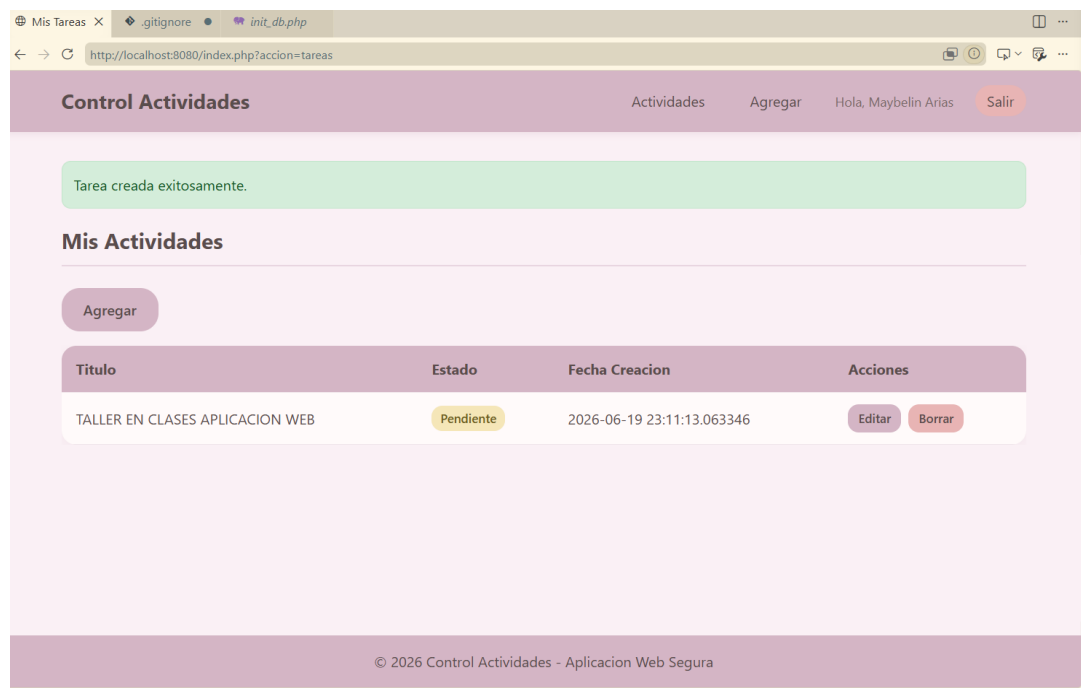
Img. 1 Página de inicio de sesión de la aplicación web "Control Actividades".

Posteriormente, se muestra la interfaz principal del sistema una vez que el usuario ha iniciado sesión correctamente. En esta sección se puede acceder a las opciones disponibles para la gestión de actividades, evidenciando el funcionamiento del módulo principal de la aplicación.



Img. 2 Vista principal del sistema de gestión de actividades después de la autenticación del usuario.

Una vez autenticado el usuario, el sistema permite gestionar las actividades registradas mediante operaciones CRUD. En la siguiente figura se muestra el listado de tareas almacenadas, así como las opciones disponibles para editar o eliminar la información según sea necesario.



Img. 3 Módulo de gestión de actividades con operaciones CRUD.

4. TABLA COMPARATIVA

A continuación se presenta una tabla comparativa de las dos tecnologías analizadas en esta práctica experimental, evaluadas bajo criterios técnicos objetivos.

Tabla 1 Tabla comparativa de tecnologías para desarrollo web: PHP 8.2 vs ASP.NET Core 8.0

Criterio	PHP 8.2	ASP.NET Core 8.0
Rendimiento	Interpretado; menor rendimiento en cálculos intensivos. PHP 8.2 mejoró con JIT.	Compilado (AOT/JIT); alto rendimiento. Superior en benchmarks (TechEmpower).
Ecosistema	Compositor, Packagist (>300k paquetes). WordPress, Laravel, Symfony.	NuGet (>300k paquetes). Blazor, MVC, Web API, gRPC, SignalR.
Seguridad por defecto	Baja: requiere implementación manual de CSRF, XSS, hashing.	Alta: Identity, AntiForgery, Razor auto-encode, Data Protection API integrados.
Curva de aprendizaje	Baja: sintaxis simple, ideal para principiantes.	Media: requiere conocimientos de C#, .NET, patrones y conceptos de Microsoft.
Plataforma	Multiplataforma (Linux, Windows, macOS).	Multiplataforma (.NET Core, Linux, Windows, macOS, Docker).
Base de datos	Conexión directa PDO (PostgreSQL, MySQL, SQLite). En este proyecto: PostgreSQL 18 con pgAdmin 4.	Entity Framework Core, Dapper, ADO.NET. Migraciones automáticas.
Comunidad y soporte	Comunidad masiva, foros, Stack Overflow. Documentación extensa.	Soporte Microsoft, comunidad activa. Documentación oficial excelente (learn.microsoft.com).

Tipado y mantenibilidad	Tipado débil; menos mantenible en proyectos grandes (PHP 7+ mejoró con tipos).	Tipado fuerte; ideal para proyectos empresariales grandes y críticos.
Costo	Gratuito y open source (The PHP Group).	Gratuito y open source (.NET Foundation).
Uso en el mercado	75%+ de sitios web (W3Techs 2025). Dominante en CMS y hosting compartido.	Creciente en empresas. Popular en APIs, microservicios y aplicaciones empresariales.

5. CONCLUSIONES

En conclusión, tanto PHP como ASP.NET Core son buenas opciones para desarrollar aplicaciones web con autenticación y operaciones CRUD. PHP es más sencillo de aprender y resulta ideal para proyectos pequeños o medianos, mientras que ASP.NET Core destaca por su mayor rendimiento, seguridad y capacidad de escalabilidad para aplicaciones empresariales.

La seguridad es un aspecto fundamental en ambos entornos. La implementación de las recomendaciones de OWASP y el uso de técnicas como los prepared statements o Entity Framework Core permiten prevenir ataques de inyección SQL. Además, el almacenamiento seguro de contraseñas mediante algoritmos de hashing garantiza una mayor protección de la información.

Finalmente, la experiencia con SQLite y PostgreSQL demostró la flexibilidad de PHP mediante PDO, permitiendo cambiar de motor de base de datos con pocos ajustes. SQLite es una alternativa adecuada para pruebas y desarrollo, mientras que PostgreSQL ofrece mejores prestaciones para entornos de producción y aplicaciones con mayor demanda.

REFERENCIAS

- [1] “PHP: Password Hashing - Manual.” Accessed: Jun. 18, 2026. [Online]. Available: <https://www.php.net/manual/en/book.password.php>
- [2] “Introduction to ASP.NET Core Framework | Coursera.” Accessed: Jun. 18, 2026. [Online]. Available: https://www.coursera.org/learn/introduction-to-aspdotnet-core-framework?utm_medium=sem&utm_source=gg&utm_campaign=b2c_latam_x_multi_ftcof_ca_reer-academy_cx_dr_bau_gg_pmax_gc_s1-v2_en_m_hyb_24-04_x&campaignid=21239613742&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&creativeid=&assetgroupid=6501905150&targetid=&extensionid=&placement=&gad_source=1&gad_campaignid=21320925518&gclid=CjwKCAjw0dPRBhAPEiwAE5vTTij02GjJDxPZmb9vpcd94DScvU4eocIGoQgBLqMS-9YCu0cvhIA3FhoCOQQQAvD_BwE
- [3] J. Garcia-Alfaro and G. Navarro-Arribas, “Prevention of cross-site scripting attacks on current web applications,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 4804 LNCS, no. PART 2, pp. 1770–1784, 2007, doi: 10.1007/978-3-540-76843-2_45.
- [4] “OWASP Top Ten Web Application Security Risks | OWASP Foundation.” Accessed: Jun. 18, 2026. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [5] “(PDF) Prevention of Cross-Site Scripting Attacks on Current Web Applications.” Accessed: Jun. 18, 2026. [Online]. Available: https://www.researchgate.net/publication/43669253_Prevention_of_Cross-Site_Scripting_Attacks_on_Current_Web_Applications

ANEXOS

Estructura del proyecto desarrollada en Visual Studio Code, evidenciando la organización de carpetas y archivos utilizados para la construcción de la aplicación web y la implementación de las diferentes capas del sistema.

